Volume 06 | Frontend Templates and JavaScript

Follow dashboard controls into the FastAPI JSON response and charts.

1. Template and browser split

`index.html` receives station metadata/default date/time and renders controls plus initial layout. The page loads dashboard JavaScript and chart library via static/template assets. `landing.html` renders marketing/preview state and `landing.js` draws an SVG preview curve, scroll-triggered train motion, reveals and mobile menu. `result.html` renders the form result and chart/travel table. FastAPI/Jinja2 supplies server context; JavaScript owns dashboard refreshes.

2. Dashboard startup and events

On DOMContentLoaded, `initControls()` finds station/date/time/slider/line/search/form elements, attaches handlers and calls `fetchPrediction`. Station change rebuilds line options based on `window.STATIONS_META`, then fetches. Slider and time input synchronize; date presets compute next weekday/Saturday/Sunday; date and line changes fetch; travel search filters the table client-side.

3. Request concurrency and race control

`fetchPrediction()` increments `predictionRequestId`, collects values, shows status/loading and POSTs JSON to `/api/predict`. If a prior response returns after a newer request, its ID no longer matches and it is ignored. This prevents stale UI overwrite, though it does not cancel earlier server computation. It checks `status === success`, updates current data and renders; network failures show a user message. `finally` hides spinner only for latest request.

4. Rendering and safe DOM usage

`renderDashboard(data)` updates station, datetime/calendar text, demand, service status, headway, train frequency and capacity cards; then updates charts, per-line breakdown and travel result list. It uses `textContent` in many display paths, which avoids interpreting returned strings as HTML. Chart instances are destroyed/recreated or resized as appropriate. Demand profile arrays and labels are API contracts; missing fields can break chart code, hence schema/contract tests are useful.

5. Search and line choices

The station selection determines valid line choices; empty line means station overview. Specific line selection changes headline recommendation, while interchange distribution remains based on configured model. Travel search filters already returned destinations by text; it does not call a new server route each keystroke.

6. Theme handling

`theme.js` toggles `data-theme`, stores preference in localStorage with try/catch, changes accessible labels/icons, and updates Chart defaults/scales if Chart.js exists. Storage can fail in locked-down contexts but theme still applies for current page. Reduced-motion preferences affect landing animations.

7. Failure paths to understand

`fetch` can fail network or receive non-success error JSON. Current code parses JSON then handles status/message, but should inspect `res.ok` and schema for all error statuses. Rapid slider changes may create many requests despite stale-response guards. Debouncing or request cancellation is an optimization if measured. DOM IDs are a coupling between template and JS.

Exercises

Trace station change through line options and request body. Explain request ID race guard. Explain why updating via textContent is safer than innerHTML for untrusted text. List API fields needed by each chart. Propose a browser contract smoke test without claiming one already exists.